

✓ Customer Purchase Behavior Analysis using Descriptive Statistics

✓ Problem Statement

Problem Statement:

Welcome to the Probability and Statistics project!   In this exciting journey, you'll get the chance to apply the concepts you've learned in probability theory and statistics to analyze a real-world dataset. This project is your opportunity to dive deep into the world of data analysis and gain practical experience with the tools and techniques you've been learning. 

Objective:

Your mission is to analyze the provided dataset containing customer information and purchasing behavior to make informed decisions. Your goal is to identify patterns, trends, and correlations that will help your company optimize its marketing efforts and increase offer acceptance rates. 

✓ About the Dataset

Here's the link to the [dataset](#)

This data was gathered during last year's campaign. Data description is as follows;

1. Response (target) - 1 if customer accepted the offer in the last campaign, 0 otherwise
2. ID - Unique ID of each customer
3. Year_Birth - Age of the customer
4. Complain - 1 if the customer complained in the last 2 years
5. Dt_Customer - date of customer's enrollment with the company
6. Education - customer's level of education
7. Marital - customer's marital status
8. Kidhome - number of small children in customer's household
9. Teenhome - number of teenagers in customer's household
10. Income - customer's yearly household income
11. MntFishProducts - the amount spent on fish products in the last 2 years
12. MntMeatProducts - the amount spent on meat products in the last 2 years
13. MntFruits - the amount spent on fruits products in the last 2 years
14. MntSweetProducts - amount spent on sweet products in the last 2 years
15. MntWines - the amount spent on wine products in the last 2 years
16. MntGoldProds - the amount spent on gold products in the last 2 years
17. NumDealsPurchases - number of purchases made with discount
18. NumCatalogPurchases - number of purchases made using catalog (buying goods to be shipped through the mail)
19. NumStorePurchases - number of purchases made directly in stores
20. NumWebPurchases - number of purchases made through the company's website
21. NumWebVisitsMonth - number of visits to company's website in the last month
22. Recency - number of days since the last purchase

✓ Task 1 - Basic CleanUp

- **Clean and preprocess the dataset (handling missing values, data types, etc.).**
- **Analyze the distribution of customer demographics (age, education, marital status) using descriptive statistics and visualizations.**

Deliverables:

- **Cleaned and Preprocessed Dataset:**

Provide a detailed report on the steps taken to handle missing values, including imputation methods used if applicable. Document the process of ensuring consistent data types for each variable, addressing any inconsistencies.

- **Summary of Basic Statistics:**

Present calculated statistics such as mean, median, variance, and standard deviation for each relevant numerical variable. Include a concise table or summary showcasing these measures for easy reference.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('/content/Superstore Marketing Data ')

print(df.head())
```

	Id	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome
0	1826	1970	Graduation	Divorced	84835.0	0	0
1	1	1961	Graduation	Single	57091.0	0	0
2	10476	1958	Graduation	Married	67267.0	0	1
3	1386	1967	Graduation	Together	32474.0	1	1
4	5371	1989	Graduation	Single	21474.0	1	0

	Dt_Customer	Recency	MntWines	...	MntFishProducts	MntSweetProducts
0	6/16/2014	0	189	...	111	189
1	6/15/2014	0	464	...	7	0
2	5/13/2014	0	134	...	15	2
3	#####	0	10	...	0	0
4	#####	0	6	...	11	0

	MntGoldProds	NumDealsPurchases	NumWebPurchases	NumCatalogPurchases
0	218	1	4	4
1	37	1	7	3
2	30	1	3	2
3	0	1	1	0
4	34	2	3	1

	NumStorePurchases	NumWebVisitsMonth	Response	Complain
0	6	1	1	0
1	7	5	1	0
2	5	2	0	0
3	2	7	0	0
4	2	7	1	0

[5 rows x 22 columns]

```
print(df.isnull().sum())
```

```
Id          0
Year_Birth  0
Education   0
Marital_Status  0
Income      24
Kidhome     0
Teenhome    0
Dt_Customer  0
Recency      0
MntWines    0
MntFruits   0
MntMeatProducts  0
MntFishProducts  0
MntSweetProducts  0
MntGoldProds  0
NumDealsPurchases  0
NumWebPurchases  0
NumCatalogPurchases  0
NumStorePurchases  0
NumWebVisitsMonth  0
Response     0
Complain     0
dtype: int64
```

▼ handling missing values by imputation technique

```
numeric_cols = ['Income', 'MntWines', 'MntFruits', 'MntMeatProducts', 'MntFishProducts',
                'MntSweetProducts', 'MntGoldProds']
for col in numeric_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

 <ipython-input-10-4e350904a520>:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting value is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df[col].fillna(df[col].median(), inplace=True)
```

```
print(df.isnull().sum())
```

 Id 0
 Year_Birth 0
 Education 0
 Marital_Status 0
 Income 0
 Kidhome 0
 Teenhome 0
 Dt_Customer 0
 Recency 0
 MntWines 0
 MntFruits 0
 MntMeatProducts 0
 MntFishProducts 0
 MntSweetProducts 0
 MntGoldProds 0
 NumDealsPurchases 0
 NumWebPurchases 0
 NumCatalogPurchases 0
 NumStorePurchases 0
 NumWebVisitsMonth 0
 Response 0
 Complain 0
 dtype: int64

```
# For date column, fill with mode
```

```
df['Dt_Customer'].fillna(df['Dt_Customer'].mode()[0], inplace=True)
```

 <ipython-input-13-6b28d8bc001e>:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting value is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df['Dt_Customer'].fillna(df['Dt_Customer'].mode()[0], inplace=True)
```

```
df.isnull().sum()
```



	0
Id	0
Year_Birth	0
Education	0
Marital_Status	0
Income	0
Kidhome	0
Teenhome	0
Dt_Customer	0
Recency	0
MntWines	0
MntFruits	0
MntMeatProducts	0
MntFishProducts	0
MntSweetProducts	0
MntGoldProds	0
NumDealsPurchases	0
NumWebPurchases	0
NumCatalogPurchases	0
NumStorePurchases	0
NumWebVisitsMonth	0
Response	0
Complain	0

dtype: int64

df.head()



	Id	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	MntFishProducts	MntSwe
0	1826	1970	Graduation	Divorced	84835.0	0	0	6/16/2014	0	189	...	111	
1	1	1961	Graduation	Single	57091.0	0	0	6/15/2014	0	464	...	7	
2	10476	1958	Graduation	Married	67267.0	0	1	5/13/2014	0	134	...	15	
3	1386	1967	Graduation	Together	32474.0	1	1	#####	0	10	...	0	
4	5371	1989	Graduation	Single	21474.0	1	0	#####	0	6	...	11	

5 rows × 22 columns

#date to datetime format

df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'], errors='coerce', format='%m/%d/%Y')

df.head()



	Id	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	MntFishProducts	MntSwe
0	1826	1970	Graduation	Divorced	84835.0	0	0	2014-06-16	0	189	...	111	
1	1	1961	Graduation	Single	57091.0	0	0	2014-06-15	0	464	...	7	
2	10476	1958	Graduation	Married	67267.0	0	1	2014-05-13	0	134	...	15	
3	1386	1967	Graduation	Together	32474.0	1	1	NaT	0	10	...	0	
4	5371	1989	Graduation	Single	21474.0	1	0	NaT	0	6	...	11	

5 rows × 22 columns

Feature Engineering

```
current_year = 2023 # Update based on current year
df['Age'] = current_year - df['Year_Birth']

#cheking for duplicate
df.duplicated().sum()

np.int64(0)

# Replace 'ID' with the actual column name representing customer ID.
# Assuming the customer ID column is named 'Id', the code should be:
df.drop_duplicates(subset='Id', keep='first', inplace=True)
```

df

	Id	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	MntSweetProducts	Mn'
0	1826	1970	Graduation	Divorced	84835.0	0	0	2014-06-16	0	189	...	189	
1	1	1961	Graduation	Single	57091.0	0	0	2014-06-15	0	464	...	0	
2	10476	1958	Graduation	Married	67267.0	0	1	2014-05-13	0	134	...	2	
3	1386	1967	Graduation	Together	32474.0	1	1	NaT	0	10	...	0	
4	5371	1989	Graduation	Single	21474.0	1	0	NaT	0	6	...	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
2235	10142	1976	PhD	Divorced	66476.0	0	1	NaT	99	372	...	48	
2236	5263	1977	2n Cycle	Married	31056.0	1	0	2013-01-22	99	5	...	8	
2237	22	1976	Graduation	Divorced	46310.0	1	0	NaT	99	185	...	5	
2238	528	1978	Graduation	Married	65819.0	0	0	2012-11-29	99	267	...	165	
2239	4070	1969	PhD	Married	94871.0	0	2	NaT	99	169	...	0	

2240 rows x 23 columns

Task 2 - Descriptive Statistics

- Calculate measures of central tendency (mean, median, mode) and measures of dispersion (variance, standard deviation) for key variables. Identify and handle outliers if necessary.

```
print(df.describe())
```

mean	5592.159821	1968.805804	52237.975446	0.444196	0.506250
min	0.000000	1893.000000	1730.000000	0.000000	0.000000
25%	2828.250000	1959.000000	35538.750000	0.000000	0.000000
50%	5458.500000	1970.000000	51381.500000	0.000000	0.000000
75%	8427.750000	1977.000000	68289.750000	1.000000	1.000000
max	11191.000000	1996.000000	666666.000000	2.000000	2.000000
std	3246.662198	11.984069	25037.955891	0.538398	0.544538

	Dt_Customer	Recency	MntWines	MntFruits	\
count	1324	2240.000000	2240.000000	2240.000000	
mean	2013-07-19 09:00:32.628398848	49.109375	303.935714	26.302232	
min	2012-07-30 00:00:00	0.000000	0.000000	0.000000	
25%	2013-01-26 00:00:00	24.000000	23.750000	1.000000	
50%	2013-07-21 12:00:00	49.000000	173.500000	8.000000	
75%	2014-01-15 00:00:00	74.000000	504.250000	33.000000	
max	2014-06-29 00:00:00	99.000000	1493.000000	199.000000	
std	NaN	28.962453	336.597393	39.773434	

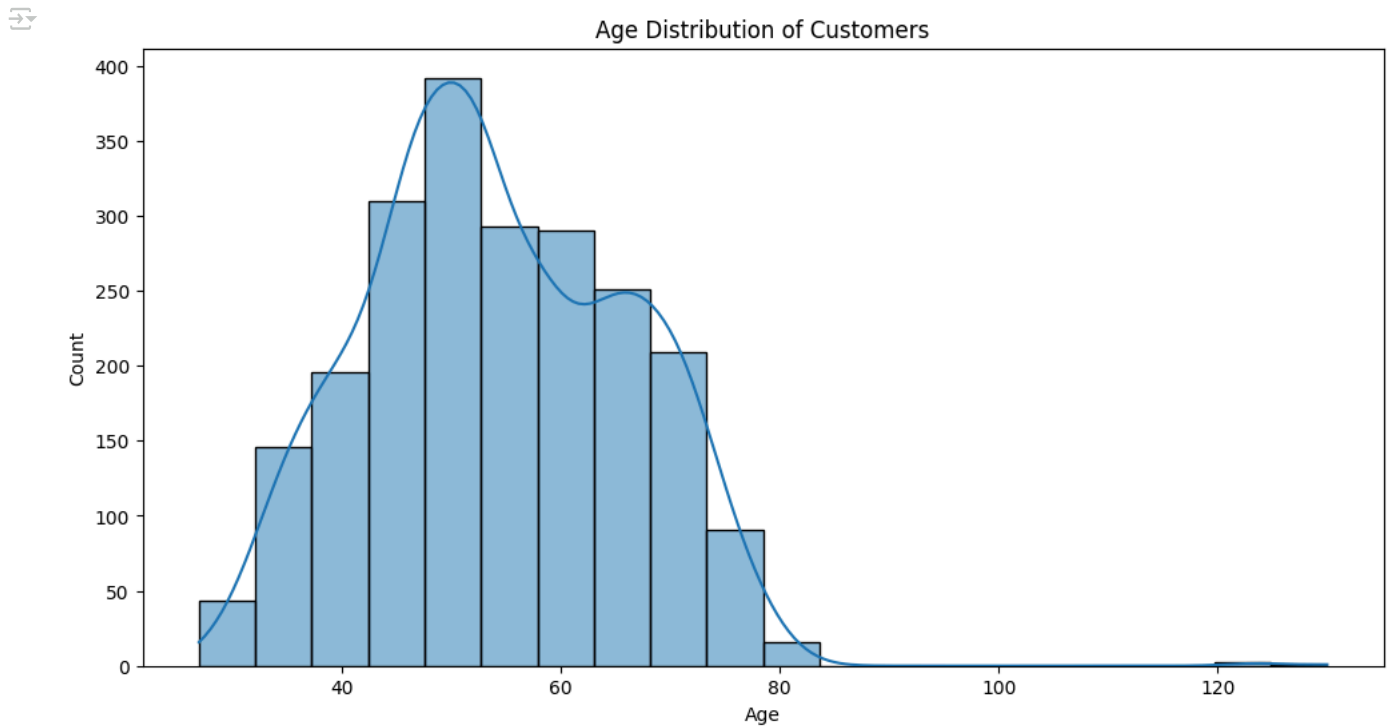
	MntMeatProducts	...	MntSweetProducts	MntGoldProds	\
count	2240.000000	...	2240.000000	2240.000000	
mean	166.950000	...	27.062946	44.021875	
min	0.000000	...	0.000000	0.000000	
25%	16.000000	...	1.000000	9.000000	
50%	67.000000	...	8.000000	24.000000	
75%	232.000000	...	33.000000	56.000000	

	NumDealsPurchases	NumWebPurchases	NumCatalogPurchases	\
count	2240.000000	2240.000000	2240.000000	
mean	2.325000	4.084821	2.662054	
min	0.000000	0.000000	0.000000	
25%	1.000000	2.000000	0.000000	
50%	2.000000	4.000000	2.000000	
75%	3.000000	6.000000	4.000000	
max	15.000000	27.000000	28.000000	
std	1.932238	2.778714	2.923101	

	NumStorePurchases	NumWebVisitsMonth	Response	Complain	\
count	2240.000000	2240.000000	2240.000000	2240.000000	
mean	5.790179	5.316518	0.149107	0.009375	
min	0.000000	0.000000	0.000000	0.000000	
25%	3.000000	3.000000	0.000000	0.000000	
50%	5.000000	6.000000	0.000000	0.000000	
75%	8.000000	7.000000	0.000000	0.000000	
max	13.000000	20.000000	1.000000	1.000000	
std	3.250958	2.426645	0.356274	0.096391	

	Age
count	2240.000000
mean	54.194196
min	27.000000
25%	46.000000
50%	53.000000
75%	64.000000
max	130.000000
std	11.984069

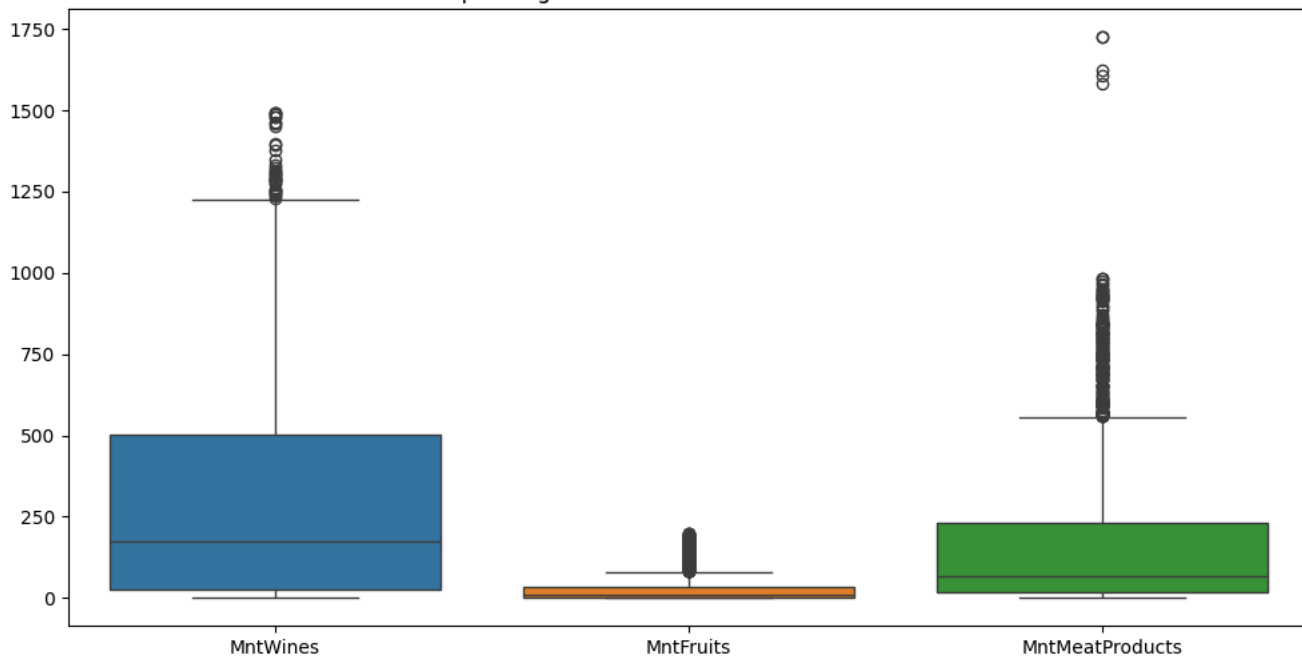
```
plt.figure(figsize=(12, 6))
sns.histplot(df['Age'], bins=20, kde=True)
plt.title('Age Distribution of Customers')
plt.show()
```



```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df[['MntWines', 'MntFruits', 'MntMeatProducts']])
plt.title('Spending Distribution on Different Products')
plt.show()
```



Spending Distribution on Different Products



Deliverables:

- Descriptive statistics that reveal the central tendencies, variations, and potential outliers in the dataset.:

✓ Task 3 - Probability Distributions

- Identify variables that could follow specific probability distributions (e.g., Binomial, Normal). Calculate probabilities and expected values based on these distributions.

```
#probability of offer acceptance
acceptance_prob = df['Response'].mean()
print(f"\nProbability of offer acceptance: {acceptance_prob:.2%}")
```



Probability of offer acceptance: 14.91%

```
#probability of high wine spending (>500)
high_wine_prob = (df['MntWines'] > 500).mean()
print(f"Probability of spending >$500 on wine: {high_wine_prob:.2%}")
```



Probability of spending >\$500 on wine: 25.22%

Deliverables:

- Determination of suitable probability distributions for relevant variables and corresponding calculated probabilities and expected values.:

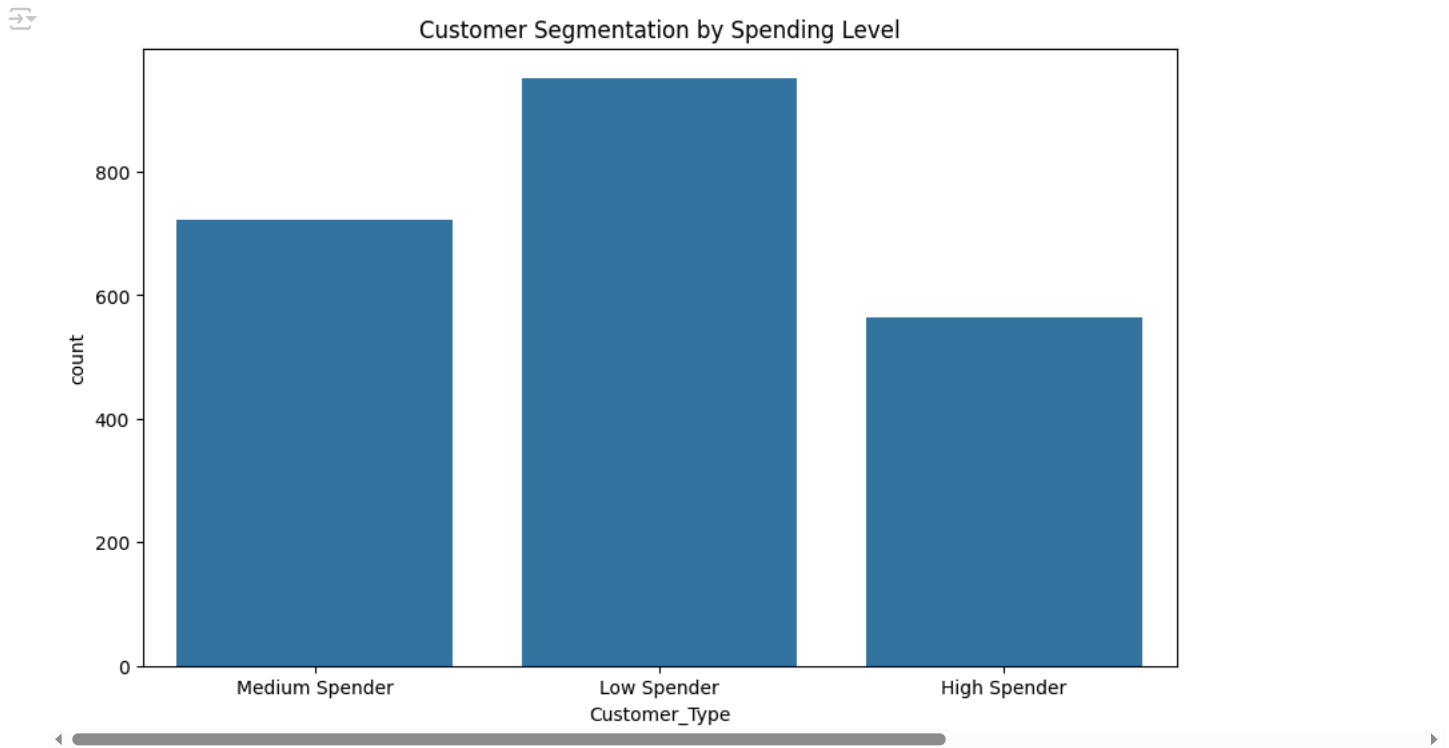
✓ Task 4: Insights and Customer Segmentation

- Explore relationships between customer characteristics and spending habits. Segment customers based on their behaviors and characteristics.

```
df['Customer_Type'] = np.where(df['MntWines'] > 500, 'High Spender',
                               np.where(df['MntWines'] > 100, 'Medium Spender', 'Low Spender'))
```

```
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Customer_Type')
```

```
plt.title('Customer Segmentation by Spending Level')  
plt.show()
```

**Deliverables:**

- Key insights regarding relationships between variables and distinct customer segments based on behaviors.

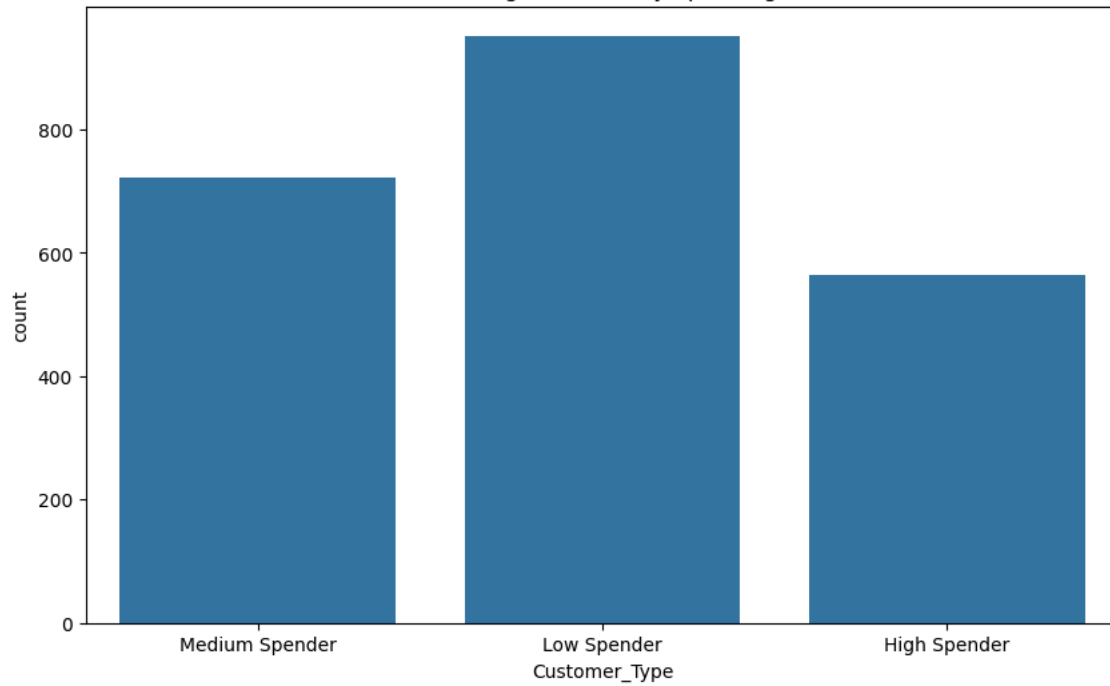
Task 5: Conclusion and Recommendations

- Create clear visualizations to showcase your findings. Use insights to make recommendations for the company based on your analysis.

```
# Visualize segmentation  
plt.figure(figsize=(10, 6))  
sns.countplot(data=df, x='Customer_Type')  
plt.title('Customer Segmentation by Spending Level')  
plt.show()
```



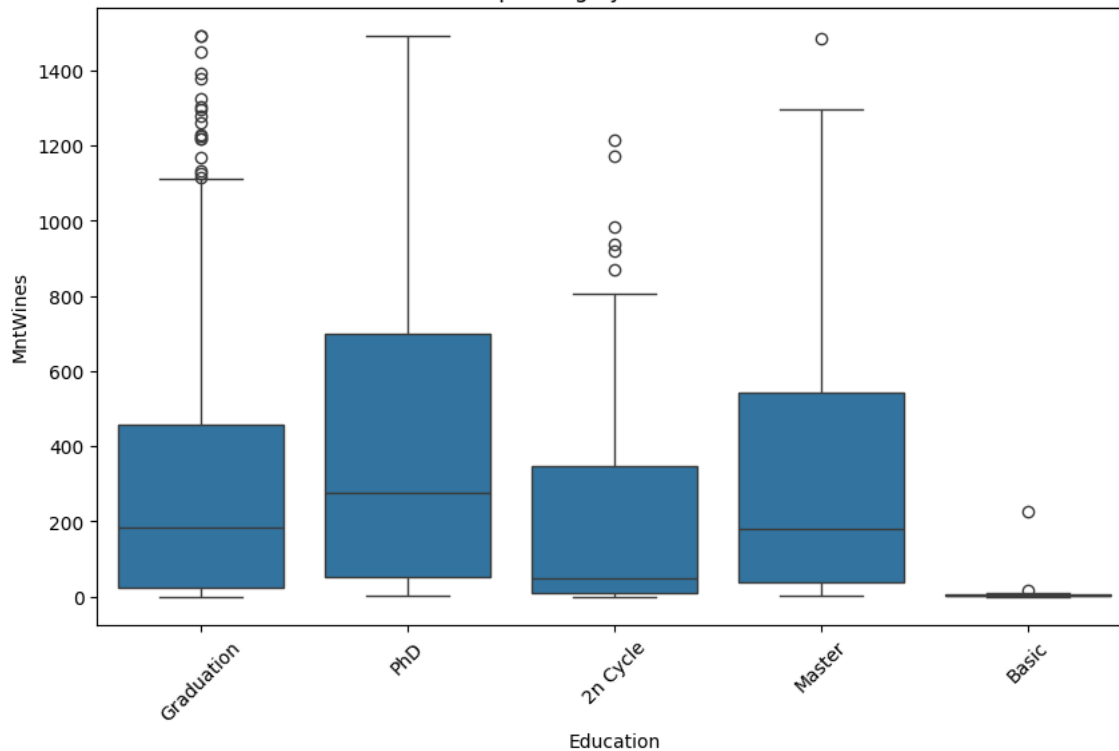

Customer Segmentation by Spending Level



```
# Education level impact
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='Education', y='MntWines')
plt.title('Wine Spending by Education Level')
plt.xticks(rotation=45)
plt.show()
```



Wine Spending by Education Level



```
# Marital status impact
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='Marital_Status', y='MntWines')
plt.title('Wine Spending by Marital Status')
```

```
plt.xticks(rotation=45)  
plt.show()
```